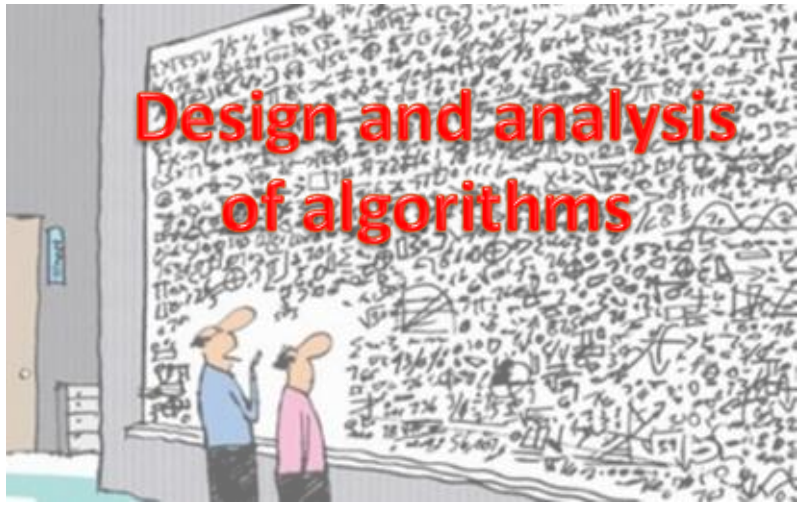


- **Amortized analysis**

Objective

- Given an algorithm, a certain operation type, e.g., insert(x), and an execution history of the system, while this operation at certain times may have a significant cost in resources, at other times it may not be as costly.
- Looking at the **worst-case** run time per operation, rather than the average per set of such operations when performed for any given input sequence, can be too pessimistic.
- The amortized analysis relates together the more costly and less costly same type operations during system histories, if these balance each other. It assigns a **fixed representative cost** to each operation, the "amortized cost", that its sum covers the total actual cost generated by these operations **at all points in time, for any system execution.**
- Reference: CLRS Ch. 17, Erik Demaine MIT slides on amortized analysis.

Amortized cost of an operation

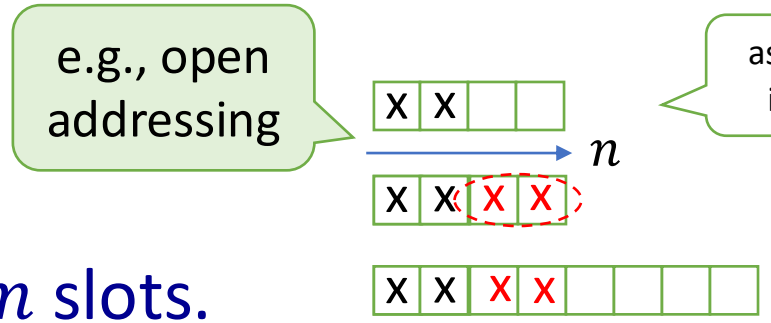
- $c(t) =$ **actual cost** of the operation at time t. It can be a function of the state of the data structure, e.g., depend on $n =$ # of elements stored at time t.
- $a(t) =$ **amortized cost** of the operation at time t. **It is what we pay out of our pocket for each operation.** Unspent money goes to a bank account.
- Suppose we show that on any trajectory of t consecutive operations, $\forall t$

$$\sum_{0 \le t' \le t} c(t') \le \sum_{0 \le t' \le t} a(t').$$

At all times, the total amount paid is enough to cover the total cost

- Then, any actual operation cost at time t is fully covered by using money from the amortized payment $a(t)$ and from the bank account using money we stored sometime in the past before time t.
- If $a(t) = O(1)$, i.e., $\forall t, a(t) \le c_a$, we say that the amortized cost of the operation is $O(1)$.

Table doubling



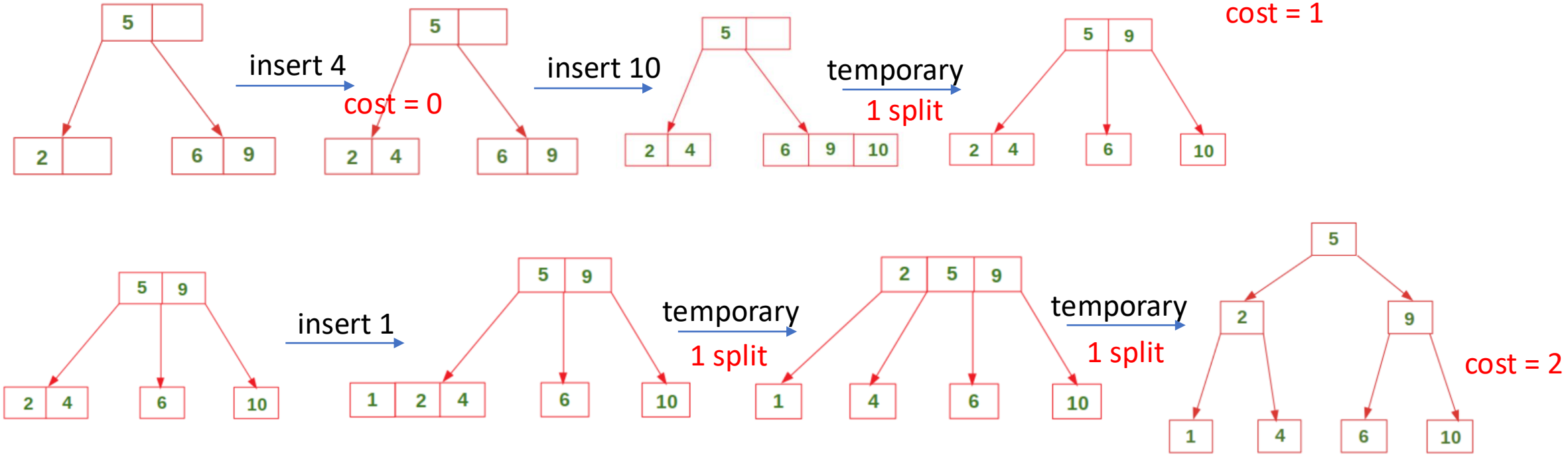
doubling cost = $\Theta(2n) = c2n$
caused by $n/2$ insertions

- n elements in a table of m slots.
- Approach: as n grows, double m when insertion occurs causing $n = m$.
- Cost of normal insertion: c_0.
- Cost of insertion when it needs to build table of size $2n$: $c_0 + 2n\,c$.
- Do we have $O(1)$ amortized cost?
- I.e., can I pay some fixed $\$c_a$ per insertion and accumulate enough funds to cover the table doubling cost when it occurs?

2-3 trees

- A 2-3 tree is a tree data structure in which every internal node (non-leaf node) has either one data element and two children or two data elements and three children.
- 2-3 trees are balanced with $O(\log n)$ for basic operations.
- Let's worry only about cost = # of splits per insertion (worse case is $O(\log n)$).

after n insertions, how many splits?



Amortized bounds for sets of operations

- Operation types $op_1, \ldots, op_k$.
- Assign each type a price: $a_1, \ldots, a_k$.
- The prices are an **amortized bound** iff, for every sequence of operations from the initial state (any mix of types) and every prefix of it:

  total actual cost of operations $\leq \sum$ (price of each operation in the prefix). (*)
- Many price vectors can satisfy (*).

Example

- For any sequence of 2-3 tree operations:
- We can say that 2-3 trees achieve
  - $O(1)$ *amortized* cost per empty tree creation,
  - $O(\log n^*)$ *amortized* cost per insert, n^* = max # of keys at any time.
  - $\emptyset$ *amortized* cost per delete.
- Why is this correct?
- Assume any sequence with t creations, i insertions, d deletions, $d \leq i$. Then,
- Actual cost is $\leq tc_1 + (i+d)c_2 \log n^* \leq tc_1 + i2c_2 \log n^* + d\,\emptyset$.

In this example we show that there are many ways to define amortized costs of operations. Amortized bounds: **Must prove that** total amortized cost always covers total actual cost.

you agree that we always have this inequality for the appropriate constants c_1, c_2?

$\leq 2i$

$=$ total amortized cost of any sequence if we use the amortized cost definition $a_{tr} = c_1, a_{in} = 2c_2, a_d = 0$.

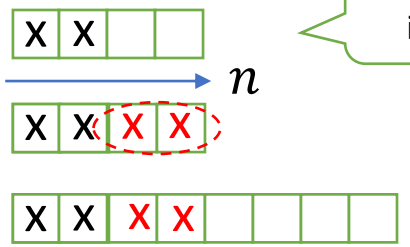
Amortization methods

- **Central concept**: allow an operation to deposit (or get back) money using some bank account besides paying for the actual cost (note: account cannot become < 0).
  - Op. i stores credit: amortized_cost$(i) >$ actual_cost(i) (part of am. cost goes to bank).
  - Op. j uses credit: amortized_cost$(j) <$ actual_cost(j) (part of act. cost is subsidized).
  - This is a general principle in all methods using amortized costs.
- Possible methods:
  - Aggregate method, accounting method, charging method, potential method.
- Different ways to construct the proof of correctness (i.e., that total amortized cost covers always total actual cost).
- We give different interpretations of how to construct the amortized costs. That helps us with the steps of the proof.

Aggregate method

- Fix an operation type **op** (for example insert). Start from the initial state and consider a possible sequence of n operations of type op (possibly interleaved with other system operations).
- Let $T(n)$ = the worst-case **total** actual cost of these n operations of type op. Then, $T(n)/n$ is the average cost per operation **in the worst case** for that n.
- **Definition.** A number $\boldsymbol{a}$ is an amortized cost of op iff $\boldsymbol{a \geq \frac{T(n)}{n}, \forall n}$.
- Equivalently, for every t, the first t operations cost at most $a \cdot t$ in total.

Table doubling cost



assume that we keep inserting elements

e.g., open addressing

doubling cost = $\Theta(2n) = c2n$
caused by $n/2$ insertions

- n elements in a table of m slots.
- Double m when an insertion causes $n = m$, cost $c2n$.
- Max of total resizing cost after k insertions: when no deletions occurred =

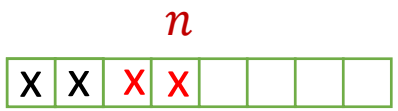
$$T(k) = c\left(2^0 + 2^1 + 2^2 + 2^3 + \cdots + 2^{\lfloor \log k \rfloor + 1}\right) = c\left(2^{\lfloor \log k \rfloor + 2} - 1\right) = \Theta(k).$$

- Worse case average *table doubling cost* per insertion is : $\frac{T(k)}{k} = \Theta(1), \forall k.$

Accounting method

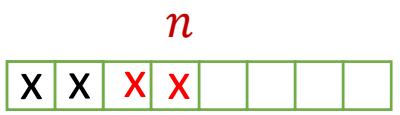
- To the normal fixed cost of the operation offer also a coin $c = \Theta(1)$.
- Compute the value of c so that total amortized cost covers total costs.

- Example: Table doubling ($n \to 2n$).



- We consider *only* the table doubling cost: 0 for normal insert, $\Theta(2n)$ for table doubling insert.
- When inserting an element, offer also a coin $c = \Theta(1)$.
  - If normal insert, this coin is stored in the bank account. If a table-doubling insert, use coins from the bank account to pay the cost.

Accounting method (cont.)



since if we allow deletions, doubling will require even more insertions

- Worse case analysis: When table must double: $n \to 2n$, there are *at least* $n/2$ coins in the bank. Use them to pay for the $\Theta(2n)$ table rebuilding cost: choose c s.t. $\Theta(2n) - 0$.

- *Amortized cost* per ("normal" or "rebuild") insert: $\Theta(1) + c$.

actual cost of normal insert

- Check that total amortized cost defined above covers at any time the total actual cost for insert on any system history (arbitrary consistent interleaving of insertions and deletions).

Charging method

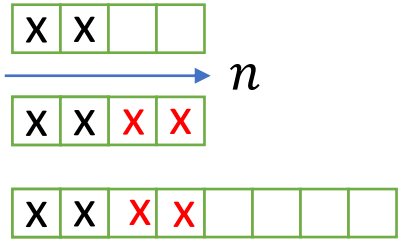
not future!

these are **responsible** for the cost

- Allow operations to charge cost *retroactively* to **past** operations.
- Amortized cost of operation =

  actual cost – total charge to past ops + total charge by future ops.
- Example: table doubling $n \to 2n$.
- When table doubles, charge $\Theta(2n)$ cost to $n/2$ inserts since last doubling.
- Each such insert is charged only once $\frac{\Theta(2n)}{n/2} = \Theta(1)$ (same constant for all n)
- Hence $\Theta(1)$ amortized cost per insert.

the maximum I had to charge any insert in any system history.



doubling cost = $\Theta(2n) = cn$
caused by $n/2$ insertions

Potential method

How to guess Φ? be smart. Its part of the solution of the problem

- Define a potential function $\Phi(\mathrm{DS}) \geq 0$.

Φ : data structure configuration → non-negative integers.

- Intuition: measures the ***potential*** **for high costs in the future** given the **current state**.
- **Amortized cost**: = **actual cost of op** + $[\boldsymbol{\Phi(\text{DS after op.}) - \Phi(\text{DS before op.})}]$ (*)

$\Delta\Phi > 0$: pay more \$ than actual cost, add to bank account

$\Delta\Phi < 0$: pay less than the actual cost, get \$ from bank account

Potential method

- Define a potential function $\Phi(\mathrm{DS}) \geq 0$ *specific* to our problem.

  Φ : data structure configuration → non-negative integers.

- Intuition: measures the ***potential*** **for high costs in the future** given the **current state**.
- **Amortized cost**: = **actual cost of op** + $[\boldsymbol{\Phi(\text{DS after op.}) - \Phi(\text{DS before op.})}]$

$$a(t) = c(t) + [\Phi(\mathrm{DS}(t)) - \Phi(\mathrm{DS}(t-1))].$$

- **Properties:**
- $\sum_{t=1}^{T} a(t) = \sum_{t=1}^{T} c(t) + [\Phi(\mathrm{DS}(T)) - \Phi(\mathrm{DS}(0))] \Rightarrow \sum_{t=1}^{T} a(t) \geq \sum_{t=1}^{T} c(t)$.

  ≥ 0 $\quad = 0$ assuming $DS(0) = \emptyset, \Phi(\emptyset) = 0$

  ⇒ total amortized costs always cover total actual costs, never overdraw the account to pay for the actual cost.

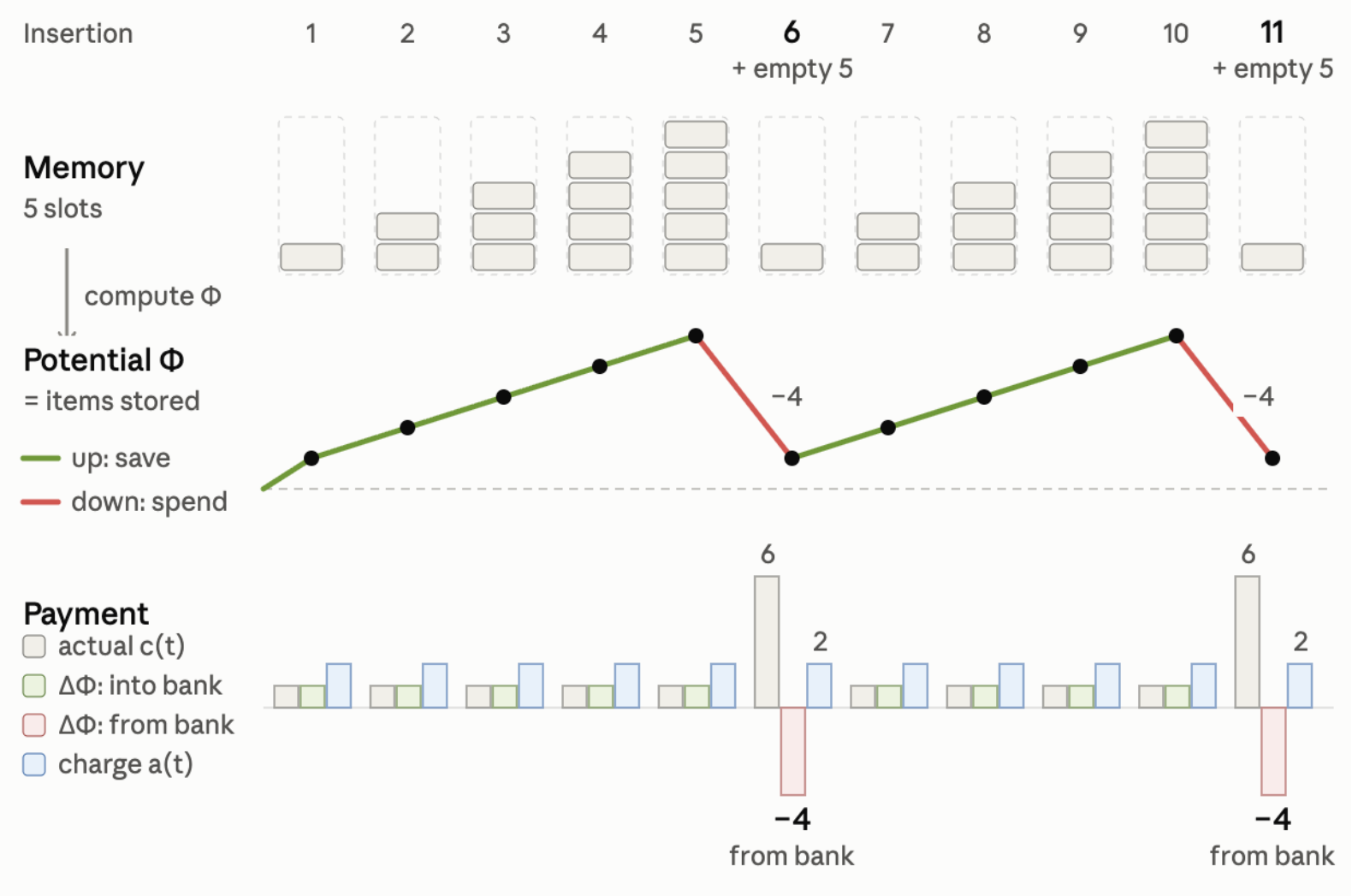
- **How to prove** $a(t) = O(1)$?
- Prove that $c(t) + \Delta\Phi(\mathrm{DS}(t)) < c_0, \forall t.$

How to guess the right Φ to satisfy this property?
Be smart. Its part of the solution of the problem

Example

- I have a total memory space of 5 slots.
- I keep inserting new numbers in empty slots.
  - Inserting a new number costs 1.
  - Emptying a slot from an old number costs 1.
- **Algorithm**: keep adding inserts to empty slots; when all full, empty all of them and insert in some empty slot.
  - The last insert more expensive!
- Use potential function $\Phi =$ # of full slots.
- Start from empty system.

Example



Charge a(t) = actual cost c(t) + ΔΦ, where ΔΦ = Φ after − Φ before insert into a free slot

Example: binary counter

000110101111
000110110000

trailing 1 bits = 4
incr. cost = 4+1

- Operation: increment.
- Increment op. cost = $\Theta(1 + \#\text{ trailing 1 bits})$.

more 1 bits → more potential for future cost

- Define $\Phi = c \cdot \#\ 1 \text{ bits in counter}$.
- $\Delta\Phi$ from increment = $c(-\#\text{ trailing 1 bits} + 1)$.
- Amortized cost of increment = actual cost + $\Delta\Phi$

  = $\Theta(1 + \#\text{ trailing 1 bits}) + c(-\#\text{ trailing 1 bits} + 1)$

  = $O(1)$ for c large enough to cancel the coefficient of $\#\text{ trailing 1 bits}$ in the Θ term.

- Last condition for this to work: $\Phi(\text{initial DS}) = 0$, assuming that we start with all zeros.

Example: insertions in 2-3 trees

Amortized split cost of insertion = actual cost + $\Delta\Phi$

- Amortize the cost of splits (= # of splits) caused by an insertion.
- $O(\log n)$ splits in worst case, we claim only $O(1)$ amortized splits per insertion.
- What causes splits? 3-nodes (with 3 children), i.e., nodes that are full (store 2 keys).
- Φ = # nodes that are full = # 3-nodes ($\triangleq$ potential for creating high cost in future).
- An insert op. causes $\Delta\Phi \leq 1 - \#\text{ splits}$,

$\Rightarrow$ amortized # of splits of an insert = # splits$+\Delta\Phi$

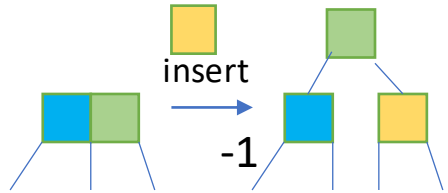
$\leq$ # splits$+(1 - \#\text{ splits})$

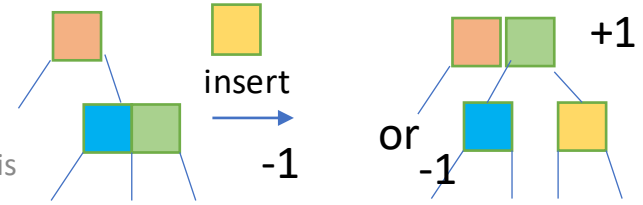
$\leq 1.$

- For the total amortized cost to cover actual cost:

  $\Phi(\text{initial DS}) = 0.$

- each split reduces Φ by 1 (breaks an existing 3-node).
- there may be a new 3-node created after an insertion, hence, to be sure add +1.





Conclusions

- The same operation on the data structure might have different cost at different times because of the different state of the data structure.
- It is not fair to always associate the worse case cost since this might be always balanced with a long period of low costs.
- Idea: define the “amortized cost” of an operation: a value that is usually constant over time. We think that operations pay this amount each time they are invoked.
- Challenge: make sure that at each time the accumulated total amortized cost covers the total actual cost.
- Why? because then we know for sure that the average actual cost per operation averaged over any time horizon and any execution is less than the amortized cost = $\Theta(1)$.
- How to guess a correct amortized cost? many different methods!